Московский Авиационный Институт
(Национальный Исследовательский Университет)

Институт №8 "Компьютерные науки и прикладная математика"
Кафедра 806 вычислительной математики и программирования

Курсовой проект по курсу
«Операционные системы»

Студент: Рязанкина Александра Олеговна
Группа: М8О–203БВ–24
Вариант: 17
Преподаватель: Соколов Андрей Алексеевич
Оценка: _____
Дата: _____
Подпись: _____

Москва, 2026.

Постановка задачи

Цель курсового проекта

• Приобретение практических навыков в использовании знаний, полученных в течении курса

• Проведение исследования в выбранной предметной области

Задание

Необходимо спроектировать и реализовать программный прототип в соответствии с выбранным вариантом. Произвести анализ и сделать вывод на основании данных, полученных при работе программного прототипа.

Требования

Исследование 2 аллокаторов памяти: необходимо реализовать два алгоритма аллокации памяти и сравнить их по следующим характеристикам:

• Фактор использования

• Скорость выделения блоков

• Скорость освобождения блоков

• Простота использования аллокатора

Каждый аллокатор памяти должен иметь функции аналогичные стандартным функциям free и malloc (realloc, опционально) или он должен быть реализован в рамках концепции std::allocator. Данный выбор производится преподавателем группы. Перед работой каждый аллокатор инициализируется свободными страницами памяти, выделенными стандартными средствами ядра. Необходимо самостоятельно разработать стратегию тестирования для определения ключевых характеристик аллокаторов памяти. При тестировании нужно свести к минимуму потери точности из-за накладных расходов при измерении ключевых характеристик, описанных выше.

Сравнение алгоритмов аллокаторов памяти (детальное описание задания описано выше). Каждый аллокатор должен обладать следующим интерфейсом (могут быть отличия в зависимости от особенностей алгоритма):

• Allocator* createMemoryAllocator(void *realMemory, size_t memory_size) (создание аллокатора памяти размера memory_size)

• void* alloc(Allocator * allocator, size_t block_size) (выделение памяти при помощи аллокатора размера block_size)

• void* free(Allocator * allocator, void * block) (возвращает выделенную память аллокатору)

Вариант 17:

Необходимо сравнить два алгоритма аллокации: алгоритм двойников и алгоритм Мак-Кьюзи-Кэрелса

Общие сведения о программе

1. **allocator_buddy.c** — первая реализация библиотеки аллокатора:
   - Реализует стратегию двойников (Buddy System), управляя списками свободных блоков для каждой степени двойки.
   - Поддерживает автоматическое расщепление больших блоков при выделении и рекурсивное слияние блоков-двойников при освобождении.
2. **allocator_mk.c** — вторая реализация библиотеки аллокатора:
   - Реализует алгоритм Мак-Кьюзи-Кэрелса
   - Использует массив списков для блоков фиксированного размера (степени двойки), что обеспечивает быстродействие O(1).
3. **tests/main_test.c** — программа для тестирования и сравнения:
   - Содержит сценарии нагрузочного тестирования.
   - Линкуется с одной из библиотек аллокаторов.
   - Замеряет время выполнения операций (alloc, free) и рассчитывает фактор использования памяти.

Общий метод и алгоритм решения.

Для реализации поставленной задачи выполнены следующие шаги:

1. Изучение алгоритмов: Изучены механизмы управления памятью, включая списковые аллокаторы и блочные схемы (Buddy System / Мак-Кьюзи-Кэрелс).

2. Разработка интерфейса: Определен единый набор функций (allocator_create, allocator_alloc, allocator_free, allocator_destroy) для обоих аллокаторов, что позволяет использовать единую программу тестирования.

3. Реализация Buddy System: Разработан модуль allocator_buddy.c, реализующий управление памятью методом двойников (Buddy System). Реализован алгоритм выделения блоков размером степени двойки через расщепление больших блоков и механизм дефрагментации путем рекурсивного слияния освободившихся блоков-двойников

4. Реализация Мак-Кьюзи-Кэрелса: Разработан модуль allocator_mk.c, реализующий управление памятью через разделение на страницы и использование отдельных списков для блоков разных размеров.

5. Разработка системы тестирования: Реализована программа main_test.c, которая:

◦ Инициализирует аллокатор на большом буфере памяти.

◦ Выполняет серии операций выделения и освобождения (10 000 итераций).

◦ Замеряет время работы с высокой точностью (CLOCK_MONOTONIC).

◦ Оценивает эффективность использования памяти (отношение полезной нагрузки к выделенному объему).

6. Сборка и анализ: Настроена система сборки CMake для автоматической компиляции и проведен сравнительный анализ аллокаторов по скорости и фрагментации.

Описание исследуемых алгоритмов

1. **Алгоритм двойников**

Принцип работы: это метод управления памятью, который делит память на блоки, размеры которых всегда являются степенями двойки (2n)

- Выделение (alloc): При запросе памяти размер округляется до ближайшей степени двойки, и алгоритм проверяет список свободных блоков этого размера. Если готового блока нет, он ищет блок большего размера (следующей степени) и рекурсивно делит его пополам, пока не получит кусок нужного объема. Одна половина отдается пользователю, а вторая («двойник») отправляется в список свободных блоков соответствующего уровня для будущих запросов.
- Освобождение (free): Когда блок освобождается, алгоритм вычисляет адрес его «двойника» и проверяет, свободен ли он. Если двойник свободен и имеет тот же размер, они объединяются (склеиваются) в один блок большего размера, который затем пытается объединиться со своим двойником на новом уровне. Этот процесс продолжается каскадно вверх, позволяя автоматически дефрагментировать память и собирать мелкие куски обратно в крупные непрерывные области

Особенности:

- Расщепление и склеивание очень быстрые.
- Память автоматически "собирается" обратно в большие куски при освобождении.

2. **Алгоритм Мак-Кьюзи-Кэрелса**

Принцип работы: Память делится на страницы (4096 байт). Каждая страница нарезается на блоки фиксированного размера, равного степени двойки (16, 32, 128, 256, 2048, 4096). Аллокатор поддерживает массив списков ("корзин") для каждого размера.

- Выделение (alloc): Размер запроса округляется до ближайшей степени двойки. Блок берется из головы соответствующего списка за константное время O(1). Если список пуст, выделяется и нарезается новая страница.
- Освобождение (free): По адресу блока вычисляется адрес страницы (выравнивание по маске). Блок возвращается в список свободных блоков этой страницы за O(1).

Особенности:

- Высокая и предсказуемая скорость работы.
- Отсутствие внешней фрагментации (блоки одного размера лежат вместе).
- Внутренняя фрагментация (потеря памяти при округлении размера до степени двойки).

Стратегия тестирования

Для определения ключевых характеристик была разработана программа тестирования (tests/main_test.c), реализующая следующие сценарии:

Обоснование подхода

1. Изоляция от ОС: Аллокаторы работают внутри заранее выделенного буфера (malloc на 10 МБ). Это позволяет измерять чистую производительность алгоритмов без учета накладных расходов на системные вызовы (mmap, sbrk).
2. Минимизация погрешности: Для замеров времени используется CLOCK_MONOTONIC, обеспечивающий высокую точность и независимость от перевода системных часов.
3. Стресс-тестирование: Используются большие объемы данных (10 000 блоков) для получения статистически значимых результатов.

Сценарии тестов

1. Тест скорости выделения (Allocation Speed):

- Выполняется 10 000 последовательных аллокаций блоков фиксированного размера (64 байта).
- Измеряется суммарное время выполнения.

2. Тест скорости освобождения (Free Speed):
    - Освобождаются все 10 000 блоков, выделенных на предыдущем этапе.
    - Измеряется суммарное время.
3. Тест на фрагментацию (Usage Factor):
    - Выполняется 5 000 аллокаций случайного размера (от 16 до 144 байт).
    - Затем делается попытка выделить один очень большой блок (1 МБ).
    - Критерий: Если большой блок успешно выделен, значит, аллокатор смог сохранить память дефрагментированной (или эффективно объединить свободные блоки). Если выделение не удалось при наличии достаточного общего объема свободной памяти — фрагментация высока.

```
alexaryaz@LAPTOP-B7R2JS7R:/mnt/c/StudyProjects/OSlabs/os_cp/build$ ./test_buddy
=== Running Tests for BUDDY Allocator ===
==================================================
Testing Allocator: Current Implementation
==================================================
[TEST 1] Allocation Speed:
  Allocated 10000 blocks of 64 bytes.
  Time taken: 0.001307 seconds
[TEST 2] Free Speed:
  Freed 10000 blocks.
  Time taken: 0.000276 seconds
[TEST 3] Fragmentation Stress Test:
  Requested allocation of 5000 random blocks.
  Successfully allocated: 5000 blocks
  Total user bytes: 396561
  Big chunk (1MB) allocation: SUCCESS (Low fragmentation)
------------------------------------------------

alexaryaz@LAPTOP-B7R2JS7R:/mnt/c/StudyProjects/OSlabs/os_cp/build$ ./test_mk
=== Running Tests for MK Allocator  ===
==================================================
Testing Allocator: Current Implementation
==================================================
[TEST 1] Allocation Speed:
  Allocated 10000 blocks of 64 bytes.
  Time taken: 0.000688 seconds
[TEST 2] Free Speed:
  Freed 10000 blocks.
  Time taken: 0.000172 seconds
[TEST 3] Fragmentation Stress Test:
  Requested allocation of 5000 random blocks.
  Successfully allocated: 5000 blocks
  Total user bytes: 396561
  Big chunk (1MB) allocation: SUCCESS (Low fragmentation)
------------------------------------------------

alexaryaz@LAPTOP-B7R2JS7R:/mnt/c/StudyProjects/OSlabs/os_cp/build$
```

Результаты тестирования

Характеристика	Buddy System (Двойники)	Мак-Кьюзи-Кэрелс (МК)
Скорость выделения	Быстро (O(log N), расщепление)	Очень быстро (O(1), готовый блок)
Скорость освобождения	Быстро (O(log N), слияние)	Очень быстро (O(1), возврат в список)
Фактор использования	Средний (потери на округление до 2^n)	Средний (потери на округление до 2^n)
Устойчивость к фрагментации	Высокая (активная дефрагментация)	Очень высокая (изоляция страниц)

Основные файлы программы

allocator_buddy.c^:

```
#include "allocator.h"
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <stdio.h>

// Минимальный порядок блока. 2^5 = 32 байта.
// Этого достаточно для заголовка (8 байт) + 2 указателя (16 байт) = 24 байта.
#define MIN_ORDER 5
#define MAX_ORDER 63

// Заголовок блока. Выравниваем до 8 байт.
typedef struct BlockHeader {
    uint8_t order;
    uint8_t is_free;
    uint8_t padding[6];
} BlockHeader;

// Структура свободного блока.
// Накладывается на память блока, включая заголовок.
typedef struct FreeBlock {
    BlockHeader header;
    struct FreeBlock* next;
    struct FreeBlock* prev;
} FreeBlock;

// Структура аллокатора
struct Allocator {
```

```
    void* memory_start;
    size_t memory_size;
    FreeBlock* free_lists[MAX_ORDER + 1];
};

// Вспомогательная функция для добавления блока в список свободных
static void list_add(Allocator* allocator, FreeBlock* block, int order) {
    block->header.is_free = 1;
    block->header.order = order;

    block->next = allocator->free_lists[order];
    block->prev = NULL;

    if (allocator->free_lists[order] != NULL) {
        allocator->free_lists[order]->prev = block;
    }

    allocator->free_lists[order] = block;
}

// Вспомогательная функция для удаления блока из списка свободных
static void list_remove(Allocator* allocator, FreeBlock* block) {
    int order = block->header.order;

    if (block->prev != NULL) {
        block->prev->next = block->next;
    } else {
        allocator->free_lists[order] = block->next;
    }

    if (block->next != NULL) {
        block->next->prev = block->prev;
    }

    block->next = NULL;
    block->prev = NULL;
}

// Вычисление логарифма по основанию 2 (округление вверх)
static int get_order(size_t size) {
    int order = 0;
    size_t s = 1;
    while (s < size) {
        s <<= 1;
        order++;
    }
    return order;
```

```
}

Allocator* allocator_create(void* realMemory, size_t memory_size) {
    if (memory_size < sizeof(Allocator) + (1 << MIN_ORDER)) {
        return NULL;
    }

    Allocator* allocator = (Allocator*)realMemory;
    allocator->memory_start = realMemory;
    allocator->memory_size = memory_size;

    for (int i = 0; i <= MAX_ORDER; i++) {
        allocator->free_lists[i] = NULL;
    }

    // Вычисляем начало управляемой кучи
    uint8_t* heap_start = (uint8_t*)realMemory + sizeof(Allocator);

    // Выравниваем начало кучи (хотя бы до 8 байт, но для buddy лучше иметь смещение 0 относительно начала блока)
    // В простой реализации будем считать смещения относительно heap_start.
    // Но нам нужно разбить доступную память на блоки степеней двойки.

    size_t available_size = memory_size - sizeof(Allocator);
    uint8_t* current_addr = heap_start;

    // Жадным образом откусываем максимально возможные куски степени 2
    while (available_size >= (1 << MIN_ORDER)) {
        int order = MAX_ORDER;
        // Ищем максимальный порядок, который помещается и выровнен
        // Для корректной работы buddy system адреса блоков должны быть кратны их размеру.
        // Относительно heap_start?
        // Если мы используем относительную адресацию для поиска двойников:
        // buddy_offset = offset ^ (1 << order)
        // То offset должен быть относительно heap_start.

        size_t offset = current_addr - heap_start;

        // Находим максимальный порядок, который влезает в available_size
        // И при этом offset кратен 2^order (для выравнивания)
        while (order > MIN_ORDER) {
            size_t block_size = (size_t)1 << order;
            if (block_size <= available_size && (offset % block_size == 0)) {
                break;
            }
            order--;
        }
```

```
        // Добавляем блок в список
        FreeBlock* block = (FreeBlock*)current_addr;
        list_add(allocator, block, order);

        size_t block_size = (size_t)1 << order;
        current_addr += block_size;
        available_size -= block_size;
    }

    return allocator;
}

void* allocator_alloc(Allocator* allocator, size_t size) {
    if (allocator == NULL || size == 0) return NULL;

    size_t needed_size = size + sizeof(BlockHeader);
    int order = get_order(needed_size);
    if (order < MIN_ORDER) order = MIN_ORDER;

    // Ищем свободный блок подходящего или большего размера
    int current_order = order;
    while (current_order <= MAX_ORDER && allocator->free_lists[current_order] == NULL) {
        current_order++;
    }

    if (current_order > MAX_ORDER) {
        return NULL; // Нет памяти
    }

    // Если нашли блок большего размера, разбиваем его
    FreeBlock* block = allocator->free_lists[current_order];
    list_remove(allocator, block);

    while (current_order > order) {
        current_order--;
        // Разбиваем на два блока порядка current_order
        FreeBlock* buddy = (FreeBlock*)((uint8_t*)block + ((size_t)1 << current_order));

        // Добавляем buddy в список свободных
        list_add(allocator, buddy, current_order);

        // block остается у нас для следующей итерации (или возврата)
        // Мы как бы спускаемся вниз по дереву, держа левую половинку
    }

    // Помечаем как занятый
    block->header.is_free = 0;
```

```
    block->header.order = order;

    return (void*)((uint8_t*)block + sizeof(BlockHeader));
}

void allocator_free(Allocator* allocator, void* ptr) {
    if (allocator == NULL || ptr == NULL) return;

    FreeBlock* block = (FreeBlock*)((uint8_t*)ptr - sizeof(BlockHeader));
    if (block->header.is_free) return; // Уже свободен?

    int order = block->header.order;
    block->header.is_free = 1;

    // Пытаемся склеить с двойниками
    uint8_t* heap_start = (uint8_t*)allocator->memory_start + sizeof(Allocator);

    while (order < MAX_ORDER) {
        size_t offset = (uint8_t*)block - heap_start;
        size_t buddy_offset = offset ^ ((size_t)1 << order);
        FreeBlock* buddy = (FreeBlock*)(heap_start + buddy_offset);

        // Проверяем, что buddy находится в пределах памяти
        if ((uint8_t*)buddy < heap_start || (uint8_t*)buddy >= (uint8_t*)allocator->memory_start + allocator->memory_size) {
            break;
        }

        // Проверяем, свободен ли buddy и имеет ли он тот же порядок
        if (!buddy->header.is_free || buddy->header.order != order) {
            break;
        }

        // Склеиваем
        list_remove(allocator, buddy);

        // Новый адрес блока - минимальный из двух (block и buddy)
        if (buddy < block) {
            block = buddy;
        }

        order++;
        block->header.order = order; // Обновляем порядок объединенного блока
    }

    list_add(allocator, block, order);
}
```

```
void allocator_destroy(Allocator* allocator) {
    // Ничего особенного делать не нужно, так как вся память внутри realMemory
}
```

allocator_mk.c:

```
#include <math.h>
#include <stdbool.h>
#include <stdint.h>
#include <string.h>

#include "allocator.h"

#define PAGE_SIZE 4096
#define MAX_ORDER 12  // 2^12 = 4096 (размер страницы)
#define MIN_ORDER 4   // 2^4 = 16 байт
#define LARGE_BLOCK_ORDER 255 // Специальный маркер для больших блоков

typedef struct PageDescriptor {
    struct PageDescriptor* next;
    struct PageDescriptor* prev; // Для двусвязного списка свободных страниц

    void* page_start;
    uint8_t order;       // Размер блоков (степень двойки) или LARGE_BLOCK_ORDER
    bool is_page_free;   // Флаг: находится ли страница в глобальном списке free_pages

    uint16_t free_count; // Для мелких блоков: кол-во свободных. Для больших: кол-во страниц.
    void* free_list_head; // Голова списка свободных блоков внутри страницы
} PageDescriptor;

struct Allocator {
    void* memory_start;
    size_t total_size;

    // buckets[i] хранит список страниц, нарезанных на блоки размера 2^i.
    // Это односвязные списки (используем next).
    PageDescriptor* buckets[MAX_ORDER + 1];

    // Список полностью пустых страниц. Двусвязный.
    PageDescriptor* free_pages_head;

    PageDescriptor* descriptors_array;
    size_t descriptors_count;
};

static uint8_t get_order(size_t size) {
    uint8_t order = 0;
    size_t s = 1;
```

```
    while (s < size) {
        s <<= 1;
        order++;
    }
    if (order < MIN_ORDER) order = MIN_ORDER;
    return order;
}

// Добавление страницы в двусвязный список свободных страниц
static void list_add_free_page(Allocator* allocator, PageDescriptor* desc) {
    desc->is_page_free = true;
    desc->order = 0;
    desc->free_count = 0;
    desc->free_list_head = NULL;

    desc->next = allocator->free_pages_head;
    desc->prev = NULL;

    if (allocator->free_pages_head != NULL) {
        allocator->free_pages_head->prev = desc;
    }
    allocator->free_pages_head = desc;
}

// Удаление страницы из двусвязного списка свободных страниц
static void list_remove_free_page(Allocator* allocator, PageDescriptor* desc) {
    if (!desc->is_page_free) return;

    if (desc->prev != NULL) {
        desc->prev->next = desc->next;
    } else {
        allocator->free_pages_head = desc->next;
    }

    if (desc->next != NULL) {
        desc->next->prev = desc->prev;
    }

    desc->next = NULL;
    desc->prev = NULL;
    desc->is_page_free = false;
}

Allocator* allocator_create(void* realMemory, size_t memory_size) {
    if (memory_size < PAGE_SIZE * 2) return NULL;

    Allocator* allocator = (Allocator*)realMemory;
```

```
    allocator->memory_start = realMemory;
    allocator->total_size = memory_size;

    for (int i = 0; i <= MAX_ORDER; i++) {
        allocator->buckets[i] = NULL;
    }

    uint8_t* data_start = (uint8_t*)realMemory + sizeof(Allocator);
    size_t num_pages = (memory_size - sizeof(Allocator)) / (sizeof(PageDescriptor) + PAGE_SIZE);

    allocator->descriptors_count = num_pages;
    allocator->descriptors_array = (PageDescriptor*)data_start;

    uint8_t* pages_base = (uint8_t*)(allocator->descriptors_array + num_pages);

    // Выравнивание на границу страницы
    size_t align_offset = (uintptr_t)pages_base % PAGE_SIZE;
    if (align_offset != 0) {
        pages_base += (PAGE_SIZE - align_offset);
    }

    allocator->free_pages_head = NULL;

    for (size_t i = 0; i < num_pages; i++) {
        PageDescriptor* desc = &allocator->descriptors_array[i];
        desc->page_start = pages_base + i * PAGE_SIZE;

        if ((uint8_t*)desc->page_start + PAGE_SIZE > (uint8_t*)realMemory + memory_size) {
            allocator->descriptors_count = i;
            break;
        }

        // Инициализируем и добавляем в список свободных
        list_add_free_page(allocator, desc);
    }

    return allocator;
}

void* allocator_alloc(Allocator* allocator, size_t size) {
    if (!allocator || size == 0) return NULL;

    // --- БОЛЬШИЕ АЛЛОКАЦИИ (> PAGE_SIZE) ---
    if (size > PAGE_SIZE) {
        size_t needed_pages = (size + PAGE_SIZE - 1) / PAGE_SIZE;

        // Ищем последовательность свободных страниц
```

```
    size_t consecutive_free = 0;
    size_t start_index = 0;

    for (size_t i = 0; i < allocator->descriptors_count; i++) {
        if (allocator->descriptors_array[i].is_page_free) {
            if (consecutive_free == 0) start_index = i;
            consecutive_free++;
        } else {
            consecutive_free = 0;
        }

        if (consecutive_free == needed_pages) {
            // Нашли! Забираем страницы.
            for (size_t k = 0; k < needed_pages; k++) {
                PageDescriptor* d = &allocator->descriptors_array[start_index + k];
                list_remove_free_page(allocator, d);
            }

            // Настраиваем первый дескриптор
            PageDescriptor* first = &allocator->descriptors_array[start_index];
            first->order = LARGE_BLOCK_ORDER;
            first->free_count = (uint16_t)needed_pages; // Храним кол-во страниц

            return first->page_start;
        }
    }
    return NULL; // Не нашли непрерывного куска
}

// --- МЕЛКИЕ АЛЛОКАЦИИ (<= PAGE_SIZE) ---
uint8_t order = get_order(size);
if (order > MAX_ORDER) return NULL; // На всякий случай

PageDescriptor* desc = allocator->buckets[order];

// Ищем страницу с местом
while (desc != NULL && desc->free_count == 0) {
    desc = desc->next;
}

// Если нет, берем новую из резерва
if (desc == NULL) {
    if (allocator->free_pages_head == NULL) return NULL;

    desc = allocator->free_pages_head;
    list_remove_free_page(allocator, desc);
```

```
        // Настраиваем страницу
        desc->order = order;
        desc->next = allocator->buckets[order]; // Вставляем в начало списка бакета
        allocator->buckets[order] = desc;

        // Нарезка
        size_t block_size = (1 << order);
        size_t blocks_per_page = PAGE_SIZE / block_size;
        desc->free_count = blocks_per_page;

        uint8_t* ptr = (uint8_t*)desc->page_start;
        for (size_t i = 0; i < blocks_per_page - 1; i++) {
            *(void**)ptr = (ptr + block_size);
            ptr += block_size;
        }
        *(void**)ptr = NULL;
        desc->free_list_head = desc->page_start;
    }

    // Выделяем блок
    void* block = desc->free_list_head;
    if (block) {
        desc->free_list_head = *(void**)block;
        desc->free_count--;
    }
    return block;
}

void allocator_free(Allocator* allocator, void* block) {
    if (!allocator || !block) return;

    // Определяем страницу по адресу
    // Т.к. страницы выровнены по PAGE_SIZE, можно попробовать найти базу.
    // Но у нас есть descriptors_array, который мапит страницы линейно.
    // Лучше найти индекс страницы.

    // Предполагаем, что block находится внутри управляемой памяти.
    // Но для Large Alloc block == page_start.
    // Для Small Alloc block внутри page.

    // Найдем дескриптор.
    // Быстрый поиск: (addr - first_page_addr) / PAGE_SIZE
    // Нам нужно знать адрес первой страницы.
    // Он хранится в descriptors_array[0].page_start

    if (allocator->descriptors_count == 0) return;
```

```
uintptr_t base_addr = (uintptr_t)allocator->descriptors_array[0].page_start;
uintptr_t block_addr = (uintptr_t)block;

if (block_addr < base_addr) return; // Чужой указатель

size_t page_idx = (block_addr - base_addr) / PAGE_SIZE;
if (page_idx >= allocator->descriptors_count) return;

PageDescriptor* desc = &allocator->descriptors_array[page_idx];

// --- ОСВОБОЖДЕНИЕ БОЛЬШОГО БЛОКА ---
if (desc->order == LARGE_BLOCK_ORDER) {
    size_t num_pages = desc->free_count;
    for (size_t i = 0; i < num_pages; i++) {
        if (page_idx + i >= allocator->descriptors_count) break;
        list_add_free_page(allocator, &allocator->descriptors_array[page_idx + i]);
    }
    return;
}

// --- ОСВОБОЖДЕНИЕ МЕЛКОГО БЛОКА ---
// Возвращаем блок в список
*(void**)block = desc->free_list_head;
desc->free_list_head = block;
desc->free_count++;

// Если страница полностью свободна, возвращаем её в глобальный резерв
size_t block_size = (1 << desc->order);
size_t max_blocks = PAGE_SIZE / block_size;

if (desc->free_count == max_blocks) {
    // Удаляем из списка bucket
    // Это O(N) для односвязного списка bucket, но списки bucket обычно короткие,
    // либо мы можем сделать bucket двусвязным.
    // Пока оставим односвязным.

    PageDescriptor** prev_ptr = &allocator->buckets[desc->order];
    PageDescriptor* curr = *prev_ptr;

    while (curr != NULL) {
        if (curr == desc) {
            *prev_ptr = curr->next; // Удаляем
            break;
        }
        prev_ptr = &curr->next;
        curr = curr->next;
    }
```

```
        // Возвращаем в free_pages
        list_add_free_page(allocator, desc);
    }
}

void allocator_destroy(Allocator* allocator) {}
```

main_test.c:

```
// ============================================================
// ТЕСТИРОВАНИЕ АЛЛОКАТОРОВ
// ============================================================
// Этот файл содержит стратегию тестирования для сравнения двух алгоритмов.
// Мы измеряем три ключевые характеристики:
// 1. Скорость выделения (Allocation Speed)
// 2. Скорость освобождения (Free Speed)
// 3. Фактор использования памяти (Fragmentation / Efficiency)

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#include "allocator.h"

#define MEMORY_SIZE (10 * 1024 * 1024)

// Функция для получения точного времени (в секундах).
// Используем CLOCK_MONOTONIC, так как оно не скачет при смене времени на
// компьютере.
double get_time() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec * 1e-9;
}

void run_tests(const char* name, Allocator* (*create_func)(void*, size_t)) {
    printf("==================================================\n");
    printf("Testing Allocator: %s\n", name);
    printf("==================================================\n");


    void* real_memory = malloc(MEMORY_SIZE);

    Allocator* alloc = create_func(real_memory, MEMORY_SIZE);
```

```
if (!alloc) {
    printf("Failed to create allocator (not enough memory?)\n");
    free(real_memory);
    return;
}
// ------------------------------------------------------------
// ТЕСТ 1: Скорость массового выделения (Allocation Speed)
// ------------------------------------------------------------
// Мы выделяем 10 000 блоков по 64 байта.
// Это имитирует создание множества мелких объектов (например, узлов
// списка).
double start = get_time();
int N = 10000;
void** ptrs = malloc(
    N * sizeof(void*));  // Массив для хранения указателей, чтобы потом освободить

for (int i = 0; i < N; i++) {
    ptrs[i] = allocator_alloc(alloc, 64);
}
double end = get_time();
printf("[TEST 1] Allocation Speed:\n");
printf("  Allocated %d blocks of 64 bytes.\n", N);
printf("  Time taken: %.6f seconds\n", end - start);

// ------------------------------------------------------------
// ТЕСТ 2: Скорость массового освобождения (Free Speed)
// ------------------------------------------------------------

start = get_time();
for (int i = 0; i < N; i++) {
    allocator_free(alloc, ptrs[i]);
}
end = get_time();
printf("[TEST 2] Free Speed:\n");
printf("  Freed %d blocks.\n", N);
printf("  Time taken: %.6f seconds\n", end - start);

// ------------------------------------------------------------
// ТЕСТ 3: Фрагментация и эффективность (Fragmentation)
// ------------------------------------------------------------
// Попробуем выделить блоки случайного размера вперемешку.
// Это самый сложный сценарий для аллокаторов.

printf("[TEST 3] Fragmentation Stress Test:\n");
int M = 5000;
size_t allocated_bytes = 0;
int success_count = 0;
```

```
    for (int i = 0; i < M; i++) {
        size_t sz = (rand() % 128) + 16;  // Случайный размер от 16 до 144 байт
        ptrs[i] = allocator_alloc(alloc, sz);
        if (ptrs[i]) {
            allocated_bytes += sz;
            success_count++;
        }
    }
    printf("  Requested allocation of %d random blocks.\n", M);
    printf("  Successfully allocated: %d blocks\n", success_count);
    printf("  Total user bytes: %zu\n", allocated_bytes);

    // Попытка выделить ОЧЕНЬ большой кусок (1 МБ) после "мусора".
    // Если память сильно фрагментирована (много дырок), это не получится,
    // даже если суммарно свободного места хватает.
    void* big_chunk = allocator_alloc(alloc, 1024 * 1024);
    if (big_chunk) {
        printf("  Big chunk (1MB) allocation: SUCCESS (Low fragmentation)\n");
        allocator_free(alloc, big_chunk);
    } else {
        printf("  Big chunk (1MB) allocation: FAILED (High fragmentation)\n");
    }

    // Чистка
    free(ptrs);
    allocator_destroy(alloc);
    free(real_memory);
    printf("-----------------------------------------------\n\n");
}

// Точка входа.
// В зависимости от флага компиляции (USE_BUDDY или USE_MK)
int main() {
#ifdef USE_BUDDY
    printf("=== Running Tests for BUDDY Allocator ===\n");
#elif defined(USE_MK)
    printf("=== Running Tests for MK Allocator ===\n");
#else
    printf("=== Running Tests for LIST Allocator (First Fit) ===\n");
#endif

    // Запускаем тесты, передавая функцию создания аллокатора
    run_tests("Current Implementation", allocator_create);

    return 0;
}
```

Вывод

В соответствии с поставленной целью и задачами курсового проекта, были реализованы два алгоритма управления памятью: двойников и алгоритм Мак-Кьюзи-Кэрелса. В ходе работы было проведено сравнение двух подходов к управлению памятью.

1. Фактор использования:

• Алгоритм двойников страдает от внутренней фрагментации (округление до степени двойки), но эффективно решает проблему внешней фрагментации за счет слияния блоков.

• Алгоритм МК также теряет память из-за внутренней фрагментации, но полностью решает проблему внешней фрагментации для фиксированных размеров.

2. Скорость работы:

• Алгоритм МК является безусловным лидером по скорости. Операции выделения и освобождения выполняются за константное время.

• Алгоритм двойников показывает высокую производительность, но уступает МК из-за затрат на расщепление и слияние блоков (логарифмическая сложность).

3. Простота использования и реализации:

• Алгоритм двойников требует работы с битовой арифметикой, но концептуально элегантен.

• МК сложнее в реализации (требует выравнивания страниц, управления массивом корзин), но обеспечивает более предсказуемое поведение в высоконагруженных системах.

Для систем общего назначения, где важна универсальность, предпочтительнее использовать алгоритм двойников. Для систем с жесткими требованиями к скорости выделения памяти может быть полезнее Мак-Кьюзи-Кэрелс